

1

Notations & Core Concepts

Empirical Risk

$$R(w) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i) \quad \Rightarrow \quad \min_w R(w)$$

Gradient Descent

$$w_k \leftarrow w_{k-1} - \eta \nabla_w R(w_{k-1})$$

Stop when $|R(w_k) - R(w_{k-1})| < \delta$ or $\|w_k - w_{k-1}\| < \delta$.

Useful Identities

$$\nabla_b (a^T b) = a \quad \nabla_a \|a\|^2 = 2a \quad \|a\|^2 = a^T a$$

2

Preprocessing

Normalization

Always apply before distance-based methods (KNN, K-Means); improves GD convergence.

Methods		
Method	Formula	Result
Z-score	$\frac{x^{(d)} - \mu_d}{\sigma_d}$	$\mu=0, \sigma=1$
Min-max	$\frac{x^{(d)} - \min_d}{\max_d - \min_d}$	$[0, 1]$
Max norm	$\frac{x^{(d)}}{\max_d}$	$[-1, 1]$

Bias term: append 1 to x_i so $D \leftarrow D + 1$.

Missing Data

Drop: loses info **Global constant / mean:** fast, ignores class **Class-conditioned mean:** best

Noise / Binning

Sort $\rightarrow$ bins (equal width *or* equal freq.) $\rightarrow$ replace by bin mean/median.

Imbalanced Data

Skewed classes $\Rightarrow$ accuracy misleading (predicting majority always gives high acc.).

Fixes

Oversampling: replicate/generate (SMOTE) minority samples **Undersampling:** remove redundant majority samples **Reweighting:** $R = \frac{1}{N} \sum_i \beta_i \ell_i, \quad \beta_i = 1 - \frac{N_k}{N}$

Text: Bag of Words (BoW)

Vocabulary of D words $\rightarrow$ count occurrences per doc $\rightarrow$ normalize $\Rightarrow x_i \in \mathbb{R}^D$.

Limits: no word order; stopwords dominate; sparse.

Images: Visual BoW

Extract patches $\rightarrow$ cluster into D visual words $\rightarrow$ count $\rightarrow$ normalize $\Rightarrow x_i \in \mathbb{R}^D$.

Data Augmentation

Add samples via label-preserving transforms $\Rightarrow$ less overfitting.

Images: flip, rotate, crop, affine transform, color jitter.

Feature Engineering & Dimensionality Reduction

3 Feature Expansion & Regularization [R/CLA]

When: data has non-linear structure but you want a linear model's tractability.

Polynomial Expansion

$\phi : \mathbb{R}^D \rightarrow \mathbb{R}^F$ maps x to higher-dim features (e.g. $[1, x_1, x_2, x_1^2, x_1x_2, x_2^2]^T$).

$$\hat{y}_i = w^T \phi(x_i), \quad w^* = (\Phi^T \Phi)^{-1} \Phi^T y$$

The model is linear in $\phi(x)$ but non-linear in x .

Cover's Theorem

A complex problem is more likely linearly separable in high-dim space (provided not densely populated).

Overfitting Risk

High-degree $\phi \Rightarrow F \gg N \Rightarrow$ overfitting. Use regularization + cross-validation to choose degree.

L2 Regularization (Ridge)

Regularized Objective

$$E(w) = \underbrace{R(w)}_{\text{fit}} + \underbrace{\lambda w^T w}_{\text{penalty}}$$

$\lambda > 0$ from cross-validation. Penalizes large weights $\Rightarrow$ smoother model.

Ridge Solution (always invertible!)

$$w^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T y$$

Multi-output: $W^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T Y$

4 Dimensionality Reduction [R/CLA/CLU]

Goal: $x_i \in \mathbb{R}^D \rightarrow y_i \in \mathbb{R}^d$ with $d \ll D$, preserving structure.

PCA — Principal Component Analysis

When: linear structure; want interpretable, fast compression.

Covariance Matrix

$$C = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})(x_i - \bar{x})^T \in \mathbb{R}^{D \times D}$$

PCA Solution

$w_{(j)}$ = j -th eigenvector of C (sorted $\lambda_1 \geq \dots \geq \lambda_d$).

$W = [w_{(1)} \dots w_{(d)}] \in \mathbb{R}^{D \times d}, \quad W^T W = I_d$

Project: $y_i = W^T (x_i - \bar{x}) \in \mathbb{R}^d$

Reconstruct: $\hat{x}_i = \bar{x} + W y_i$

Choose d

Retain $\geq 90\%$ variance: $\sum_{j=1}^d \lambda_j \geq 0.9 \sum_{k=1}^D \lambda_k$

PCA minimizes reconstruction error

$$e_i = \|\hat{x}_i - x_i\|^2$$

The d leading eigenvectors minimize total reconstruction error.

Limitation: only captures linear structure.

Kernel PCA

When: non-linear structure in data.

Map $x_i \mapsto \phi(x_i)$, apply PCA in feature space. By representer theorem, eigenvectors of covariance lie in span of $\{\phi(x_i)\}$, so:

$$Ka = \lambda Na, \quad K_{ij} = k(x_i, x_j)$$

$$y_i^{(j)} = \sum_{l=1}^N a_l^{(j)} k(x_i, x_l)$$

Center kernel: $\tilde{K} = K - \mathbf{1}_N K - K \mathbf{1}_N + \mathbf{1}_N K \mathbf{1}_N$

Limitation vs PCA: no inverse mapping (can't reconstruct $\hat{x}$ from y).

Autoencoders

When: non-linear structure; need encoder & decoder; large data.

Autoencoder

Encoder: $x \rightarrow y$ (latent representation)

Decoder: $y \rightarrow \hat{x}$

Goal: $\hat{x} \approx x$, trained by minimizing $\frac{1}{N} \sum_i \|\hat{x}_i - x_i\|^2$ with SGD.

Variants

Denoising: add noise to input, train to reconstruct clean version

Sparse: add ℓ_1 penalty on latent code $\Rightarrow$ sparse activations

vs PCA: autoencoder is non-linear, more powerful; has both encoder & decoder; non-convex optimization.

Applications: compression, anomaly detection, generative models.

Feature Engineering & Dimensionality Reduction • Page 2 • [REG] Regression [CLA] Classification [CLU] Clustering

Models	.
5 Linear Regression [REG]	
<i>When:</i> output is continuous; relationship is approximately linear. <i>Output:</i> $\hat{y}_i \in \mathbb{R}$.	
Model & Loss (MSE)	
$\hat{y}_i = w^T x_i \quad R(w) = \frac{1}{N} \sum_{i=1}^N (w^T x_i - y_i)^2$	
Closed-Form Solution	
$\nabla_w R = 0 \Rightarrow w^* = (X^T X)^{-1} X^T y$ If $X^T X$ not invertible: $w^* = X^\dagger y$ (pseudo-inverse). Multi-output ($C > 1$): $W^* = X^\dagger Y$	
Prediction	
$\hat{y} = w^{*T} x \quad (C=1) \quad \hat{y} = W^{*T} x \quad (C>1)$	
Invertibility	
$X^T X$ invertible $\Leftrightarrow$ columns of X linearly independent (X full rank). Otherwise use pseudo-inverse $X^\dagger$.	
<i>Pros:</i> exact closed-form, fast, interpretable. <i>Cons:</i> only linear relationships.	
6 Logistic Regression (Binary) [CLA]	
<i>When:</i> binary classification ($y_i \in \{0, 1\}$); approximately linearly separable. <i>Output:</i> $\hat{y}_i = p(\text{class} = 1 \mid x_i) \in (0, 1)$.	
Model	
$\hat{y}_i = \sigma(w^T x_i) = \frac{1}{1 + e^{-w^T x_i}}$	
Binary Cross-Entropy Loss	
$R(w) = - \sum_{i=1}^N \left[y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i) \right]$	
Gradient (no closed form $\Rightarrow$ GD)	
$\nabla_w R(w) = \sum_{i=1}^N (\hat{y}_i - y_i) x_i$	

Prediction	
$\hat{y} = \begin{cases} 1 & \sigma(w^{*T} x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$ Threshold 0.5 can be adjusted (e.g. raise for high precision).	
7 Multi-Class Classification [CLA]	
<i>When:</i> $C > 2$ classes. Labels as one-hot: $y_i \in \{0, 1\}^C$. Linear (Least Squares) $\min_W \sum_i \ W^T x_i - y_i\ ^2 \Rightarrow W^* = X^\dagger Y$ $\hat{y}_i = W^{*T} x_i \in \mathbb{R}^C, \quad k = \arg \max_j \hat{y}_i^{(j)}$	
Softmax (Logistic)	
Softmax Output	
$\hat{y}^{(k)}(x) = \frac{\exp(w^{(k)T} x)}{\sum_{j=1}^C \exp(w^{(j)T} x)} \approx p(\text{class} = k \mid x)$	
Multi-Class Cross-Entropy	
$R(W) = - \sum_{i=1}^N \sum_{k=1}^C y_i^{(k)} \ln \hat{y}_i^{(k)}$	
Gradient & Prediction	
$\nabla_W R = \sum_{i=1}^N x_i (\hat{y}_i - y_i)^T \quad k = \arg \max_j \hat{y}_i^{(j)}$	
8 SVM — Support Vector Machines [R/CLA]	
<i>When:</i> binary classification; want maximum-margin boundary; works well in high dim. <i>Output:</i> $\hat{y} \in \{-1, +1\}$.	
Key Concept: Margin	
Distance between the hyperplane $w^T x + b = 0$ and the closest points. $\text{margin} = \frac{2}{\ w\ } \quad (\text{maximize} \Leftrightarrow \text{minimize } \ w\)$ Support vectors: the points defining the margin.	
Prediction	
$\hat{y} = \text{sign}(w^{*T} x + b)$	
9 k-Nearest Neighbors (KNN) [R/CLA]	
<i>When:</i> non-linear decision boundaries; small dataset; no training phase needed. <i>Output:</i> class label or continuous value.	

Distances	
Euclidean: $\sqrt{\sum_d (x_i^{(d)} - x_j^{(d)})^2}$	
Classification	
Find k nearest neighbors $\Rightarrow$ majority vote .	
Regression	
$\hat{y} = \frac{\sum_i w_i y_i}{\sum_i w_i} \quad w_i = \frac{1}{d_i} \text{ (inv. dist.)}$	
Hyperparameter k	
$k = 1$: very flexible, overfitting risk k large : smooth, underfitting risk Choose k by cross-validation. Use odd k for 2 classes.	
<i>Cons:</i> no training but slow prediction $O(ND)$; sensitive to scale $\Rightarrow$ normalize first! ; sensitive to outliers.	
10 Kernel Methods [R/CLA]	
<i>Why:</i> when ϕ is very high-dim or infinite-dim, computing $\phi(x)$ is infeasible. We only need dot products $\phi(x_i)^T \phi(x_j) = k(x_i, x_j)$.	
Kernel Function	
$k(x_i, x_j) = \phi(x_i)^T \phi(x_j) \quad (\text{implicit } \phi)$ Polynomial: $(x_i^T x_j + c)^d$	
Kernel Matrix & Vector	
$K \in \mathbb{R}^{N \times N}, \quad K_{ij} = k(x_i, x_j) \quad (\text{symmetric, PSD})$ $k(X, x) = (k(x_1, x), \dots, k(x_N, x))^T \in \mathbb{R}^N$	
Kernel Ridge Regression	
Reparametrize $w = \Phi^T \alpha$, solve for $\alpha \in \mathbb{R}^N$: $\alpha^* = (K + \lambda I_N)^{-1} y$ $\hat{y} = y^T (K + \lambda I)^{-1} k(X, x)$ Multi-output: $\hat{y} = Y^T (K + \lambda I)^{-1} k(X, x)$	
Kernel Selection	
Choose kernel and hyperparams (d, σ, λ) by cross-validation .	
<i>Kernel vs explicit ϕ:</i> kernel is $O(N^2)$ in space vs $O(NF)$	
11 Neural Networks / MLP [R/CLA]	
<i>When:</i> large datasets; complex non-linear structure; learns features automatically. <i>Output:</i> any type depending on final activation.	

Architecture (Forward Pass)	
Layers	
$z_{(1)} = f(W_{(1)}^T x) \quad z_{(l)} = f(W_{(l)}^T z_{(l-1)})$ $\hat{y} = W_{(L)}^T z_{(L-1)} \quad (\text{or softmax for classification})$	
Activation Functions	
ReLU: $\max(0, z)$ — hidden layers (most common) Sigmoid: $\frac{1}{1+e^{-z}}$ — binary output Softmax: $\frac{e^{z^{(k)}}}{\sum_j e^{z^{(j)}}}$ — multi-class output	
Universal Approximation	
Any continuous function can be approximated by a linear combination of translated/scaled ReLU (or sigmoid) functions.	
Training: Backpropagation	
Loss Functions	
Regression: $R(W) = \frac{1}{N} \sum_i \ \hat{y}_i - y_i\ ^2$ Classification: $R(W) = -\frac{1}{N} \sum_i \sum_k y_i^{(k)} \ln \hat{y}_i^{(k)}$	
Backprop Algorithm	
- Forward pass: compute all $z_{(l)}$ and $\hat{y}$. - Compute output sensitivity $\delta_{(L)}^{(k)} = \frac{\partial \ell_i}{\partial a_{(L)}^{(k)}}$. - Propagate backward: $\delta_{(l)} \rightarrow \delta_{(l-1)}$ via chain rule. - Gradient: $\frac{\partial \ell_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} z_{(l-1)}^{(j)}$.	
SGD & Variants	
Mini-Batch SGD	
$W_k \leftarrow W_{k-1} - \eta \sum_{b=1}^B \nabla \ell_{(b,k)}(W_{k-1})$ B = batch size. Full GD: $B = N$; SGD: $B = 1$; mini-batch: $1 < B < N$.	
SGD with Momentum	
$\nu \leftarrow \gamma \nu + \eta \nabla R(W) \quad W \leftarrow W - \nu$ $\gamma > 0$ adds inertia; accelerates if gradient is consistent.	
<i>Hyperparams:</i> learning rate η , batch size B , depth L , width M , activation. <i>Overfitting:</i> use dropout, weight decay (L2), data augmentation, early stopping.	
12 Convolutional Neural Networks [REG/CLA]	
<i>When:</i> image/spatial data; translation-invariant features; large datasets.	

Output: any type depending on final layer (same as MLP).

Architecture
Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow \dots \rightarrow$ Flatten $\rightarrow$ FC layers $\rightarrow$ output Conv layer: learns spatial filters; shares weights $\Rightarrow$ fewer params Pooling: reduces spatial size; max pooling keeps strongest activation FC layer: standard MLP at the end for final prediction
Hyperparameters
Filter size: e.g. 3×3 , 5×5 Stride: step size of the sliding filter Padding: preserve spatial dims (same) or reduce (valid) Depth: number of filters per layer

Key advantage over MLP: weight sharing + local connectivity $\Rightarrow$ translation invariance, far fewer parameters for image inputs.

Training: backprop + SGD, same as MLP.

13 K-Means Clustering [CLU]

When: unsupervised; want to partition N unlabeled points into K groups.

Algorithm
1. Initialize K centroids $\mu_1, \dots, \mu_K$ randomly. 2a. Assign: $c_i = \arg \min_k \ x_i - \mu_k\ ^2$ for all i . 2b. Update: $\mu_k = \frac{1}{ C_k } \sum_{i:c_i=k} x_i$. 3. Repeat until assignments don't change.

Key Caveats
Convergence: guaranteed, but may reach local minimum (not global) Initialization: run multiple times with different random inits Scale: always normalize features — K-Means uses Euclidean distance! Choose K: elbow method on within-cluster variance

Complexity: $O(NKD)$ per iteration.
Limitation: assumes spherical clusters; sensitive to outliers.

14 Evaluation — Regression

Regression Metrics

MSE: $\frac{1}{N} \sum (y_i - \hat{y}_i)^2$ — penalizes large errors

RMSE: $\sqrt{\text{MSE}}$ — same unit as y ; more interpretable

MAE: $\frac{1}{N} \sum |y_i - \hat{y}_i|$ — robust to outliers

MAPE: $\frac{1}{N} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right|$ — relative; undef. if $y_i=0$

Interpretation

MSE≈ 0: near-perfect predictions **RMSE:** error in the unit of y , e.g. RMSE=5€ means avg error of 5€ **MAE vs MSE:** MAE if outliers shouldn't dominate; MSE if large errors are critical

15 Evaluation — Classification

Confusion Matrix (binary)		
	Pred Pos	Pred Neg
True Pos	TP	FN (type II)
True Neg	FP (type I)	TN
$P = TP + FN \quad N = FP + TN$		

Metrics

Accuracy: $\frac{TP+TN}{TP+TN+FP+FN}$ **Precision:** $\frac{TP}{TP+FP}$

Recall: $\frac{TP}{TP+FN}$ **FPR:** $\frac{FP}{FP+TN}$ **F1:** $2 \cdot \frac{\text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$

Interpretation — Goal → Metric

Reduce FP: ↑ Precision (spam filter, recommendations) **Reduce FN:** ↑ Recall (cancer detection, fraud) **Balance FP & FN:** ↑ F1 (imbalanced classes) **Reduce FPR:** ↓ FP/(FP+TN): fewer false alarms on negatives **Imbalanced data:** F1 or weighted avg; *Accuracy misleading!*

Multi-class: $C \times C$ confusion matrix. Compute precision/recall per class (one-vs-all) then average.

- **Macro avg:** equal weight per class
- **Weighted avg:** weight by class size; use if imbalanced

16 Evaluation — Clustering

Without labels (unsupervised):

Inertia (Within-cluster variance)

$$W = \sum_{k=1}^K \sum_{i: c_i=k} \|x_i - \mu_k\|^2$$

Lower W = tighter clusters.

Choose K — Elbow Method

Plot W vs K . Choose K at the "elbow" where gain diminishes.

With labels (supervised): training is unsupervised but if ground truth labels are available, Accuracy, Precision, Recall, F1 apply identically to classification.

17 Overfitting, Underfitting & CV

Bias-Variance

Underfitting: high error on train & test (model too simple) **Overfitting:** low train error, high test error (model too complex)

Cross-Validation Strategies

Val. set: 80/20 split; fast but estimate depends on random split **K-fold:** split in K parts, rotate; $K=5$ or 10 is standard **LOOCV:** $K = N$; exact but very expensive

Rule: never use test set to choose hyperparameters — use validation/CV only.

18 Model Comparison Summary

Model	T	Train	Hyper
Lin. Reg.	REG	closed form	—
Log. Reg.	CLA	GD	η
Softmax	CLA	GD	η
SVM	REG/CLA	QP solver	C
KNN	REG/CLA	none	k
Ridge	REG	closed form	λ
Kernel Ridge	REG/CLA	closed form	λ, k
MLP	REG/CLA	SGD+backprop	η, L, M
PCA	DR	eigendecomp	d
Kernel PCA	DR	eigendecomp	d, k
Autoencoder	DR	SGD	arch
K-Means	CLU	iterative	K

Model	Key trait
Lin. Reg.	closed form $w^* = (X^T X)^{-1} X^T y$; linear only
Log. Reg.	sigmoid output; probabilistic; no closed form
Softmax	generalizes logistic to C classes
SVM	max-margin; C large → tight (overfit)
KNN	no training; slow inference; normalize!
Ridge	L2 penalty $\lambda \ w\ ^2$; always invertible
Kernel Ridge	implicit ϕ ; $\alpha^* = (K + \lambda I)^{-1} y$
MLP	learns ϕ automatically; non-convex
PCA	linear; max variance; has reconstruction
Kernel PCA	non-linear; no inverse (no reconstruction)
Autoencoder	non-linear; has encoder & decoder
K-Means	unsupervised; local min; normalize!